Infrawatch — Architecture Overview

Infrastructure monitoring platform · Air-gap capable · Generated 2026-03-28

End-to-End Data Flow

Go Agent → gRPC / mTLS → Ingest Service (Go) → Redpanda Topics → Go Consumers → PostgreSQL / TimescaleDB → Next.js Web App

LAYER 1 — FIELD AGENTS

Go Agent (Apache 2.0)

Single binary · No runtime deps · Runs on monitored host

- Generates keypair on first run
- Registers with Ingest → Pending state
- Admin approves in UI → signed JWT issued
- Pushes metrics/events over gRPC + mTLS (port 443)
- Falls back to HTTPS polling if gRPC blocked
- Self-update from server (air-gap safe)

Agent Check Capabilities

New features are check definitions, not agent code changes

- **shell** — run script, capture stdout/stderr/exit
- **file** — read / hash / stat a file
- **port** — TCP/UDP connectivity check
- **process** — check if process is running
- **http** — internal HTTP health check
- **metric** — CPU / memory / disk / network

Agent Identity Model

mTLS + JWT · Offline token validation

- Keys generated on-host, never transmitted
- Server issues JWT bound to agent public key
- Tokens rotate automatically
- Self-update: signed binary, verified before hot-swap
- Rollback on failure

gRPC + mTLS (port 443) — push model, agent initiates

LAYER 2 — INGEST SERVICE

apps/ingest — Go gRPC Service

Accepts agent connections · Validates identity · Publishes to queue

- Validates agent JWT + mTLS certificate on every call
- Deserialises protobuf messages (proto/agent/v1/)
- Routes to appropriate Redpanda topic
- Handles agent registration and approval workflow
- Responds to agent version-check requests (self-update)
- Scales horizontally in HA deployment

proto/agent/v1/

Canonical protobuf definitions

- Shared between agent (Go) and ingest (Go)
- TypeScript types generated → packages/proto-ts/
- Single source of truth for all agent messages

Publish to topics: metrics.raw · events.raw · alerts.pending · agent.status

LAYER 3 — MESSAGE QUEUE (ABSTRACTED)

metrics.raw

- CPU, memory, disk, network samples
- Raw from agent — not aggregated

events.raw

- State change events from checks
- Process start/stop, file changes

alerts.pending

- Alert candidates for rule evaluation
- Consumed by alerts consumer

agent.status

- Agent heartbeat and health
- Registration and approval events

Queue abstraction: implementation swapped via config — *small* (<50 hosts): in-process Go channels + WAL · *standard*: Redpanda single node · *ha*: Redpanda cluster

Consumed by independent Go binaries

LAYER 4 — QUEUE CONSUMERS (GO)

consumers/metrics/

Reads: metrics.raw

- Writes time-series data to TimescaleDB
- Applies aggregation / downsampling

consumers/alerts/

Reads: alerts.pending

- Evaluates alert rules against incoming data

consumers/events/

Reads: events.raw · agent.status

- Appends to events spine in PostgreSQL
- Triggers webhooks on state changes

- Independently scalable

- Writes alert instances to PostgreSQL
- Triggers notification routing

- Updates agent status tables

Write to PostgreSQL + TimescaleDB

LAYER 5 — DATABASE

PostgreSQL + TimescaleDB

Drizzle ORM · Schema in apps/web/lib/db/schema/

- **Multi-tenancy** — every row has `organisation_id`
- **Soft deletes** — `deleted_at` on every table, never hard delete
- **Tags** — flexible `key:value` on any resource via join table
- **Event spine** — significant state changes append to `events`
- **Status as state machine** — current status + history table
- **JSONB escape hatch** — `metadata jsonb` on every major table
- IDs: cuid2 · Timestamps: UTC with timezone

Schema Domains

One file per domain in schema/

- organisations · users · teams
- agents · hosts
- metrics (TimescaleDB hypertable)
- certificates
- alerts · alert_rules · alert_instances
- events (append-only spine)
- api_keys · service_accounts
- resource_tags (join table)

Server Actions (Next.js) query via Drizzle ORM

LAYER 6 — WEB APPLICATION (APPS/WEB)

Next.js 15 App Router

TypeScript strict · shadcn/ui · Tailwind CSS

- Server components for initial data / metadata
- Client components for interactive dashboards
- TanStack Query for client-side data fetching
- Server Actions for all mutations (Zod validated)
- React Hook Form + Zod for forms
- Recharts (wrapped) for all charts

Auth — Better Auth

Community + Enterprise tiers

- Email + password · TOTP MFA · API keys
- LDAP / Active Directory (community tier)
- SAML 2.0 · OIDC (enterprise)
- RBAC: super_admin · org_admin · engineer · read_only · agent
- Licence: offline JWT · signed against bundled public key · no phone-home

Component Structure

components/ directory

- **ui/** — shadcn primitives (do not modify)
- **charts/** — Recharts wrappers
- **hosts/** — domain components
- **certificates/**
- **alerts/**
- **shared/** — PageHeader, DataTable, StatusBadge

Deployment Profiles (deploy/)

Single (docker-compose.single.yml)

- PostgreSQL
- In-process queue (Go channels + WAL)
- Single Ingest instance
- Single Next.js instance
- < 50 hosts

Standard (docker-compose.standard.yml)

- PostgreSQL
- Redpanda single node
- Single Ingest instance
- Single Next.js instance
- 50–500 hosts

HA (docker-compose.ha.yml / Helm)

- PostgreSQL primary + replica
- Redpanda cluster
- Multiple Ingest instances
- Multiple Next.js instances
- HAProxy load balancer
- 500+ hosts

ENTERPRISE BOUNDARY

apps/web/enterprise/ — Proprietary

Compiled in via feature flag · Source-available · Licence gated

- SAML 2.0 / OIDC SSO
- Audit log
- Advanced RBAC
- Compliance packs
- White-label

Air-Gap Support

No internet dependency for any core feature

- All images bundleable: `docker save | gzip`
- `deploy/scripts/airgap-bundle.sh`
- No CDN — all assets served locally
- Agent updates from Infrawatch server, not internet
- Licence validated offline (bundled public key)

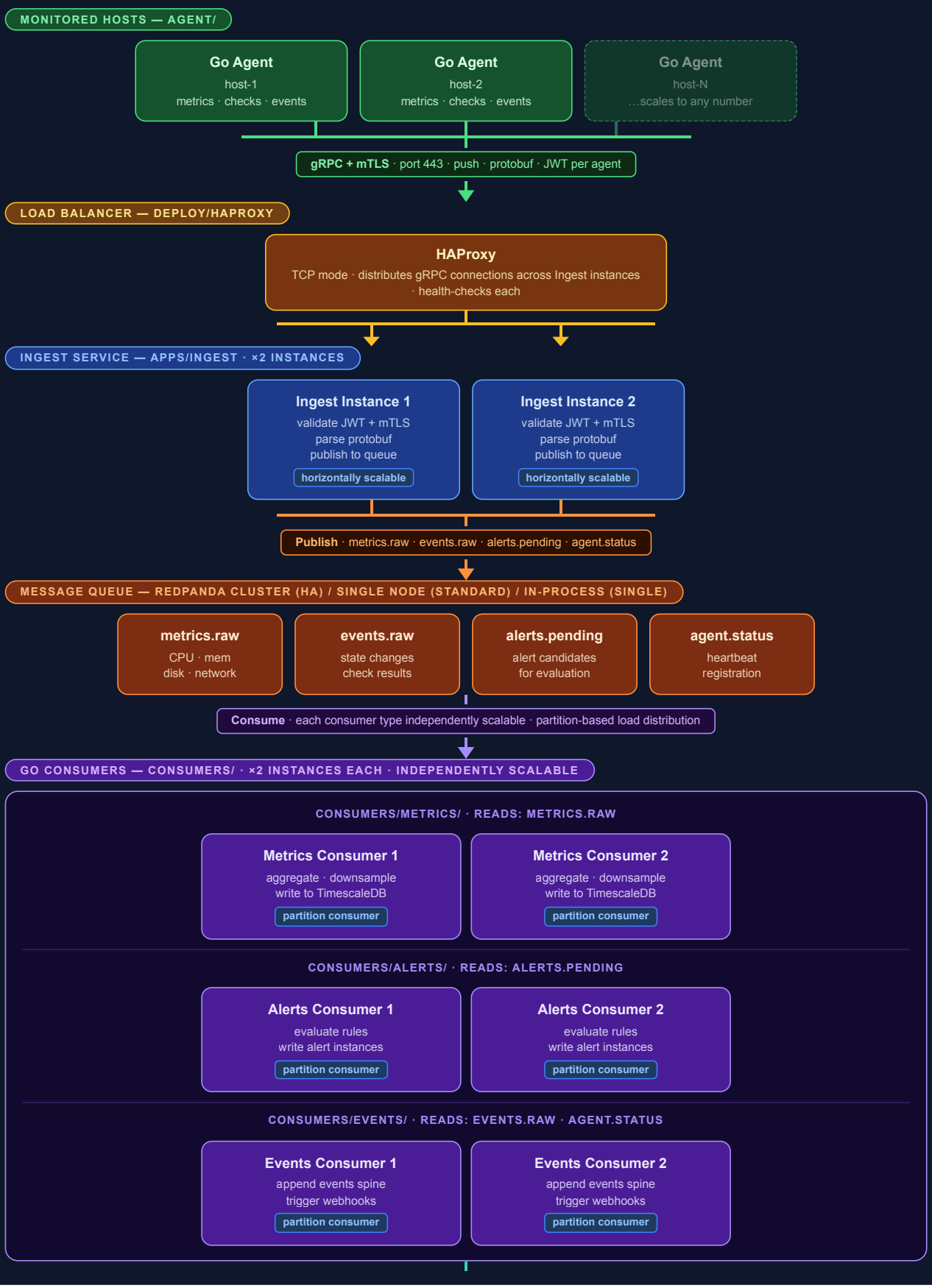
Roadmap Phases

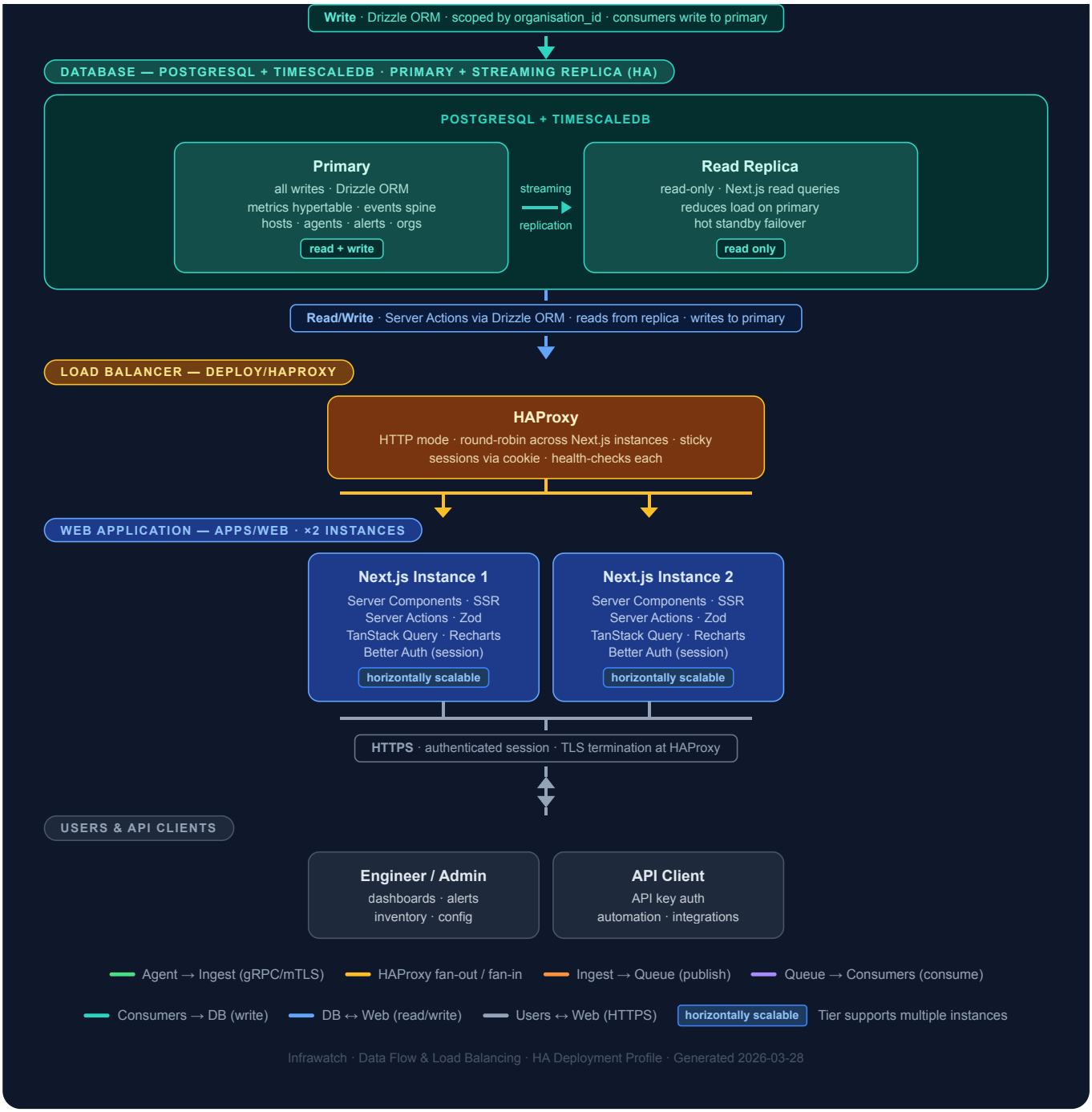
Each phase independently deployable

- **0** Foundation (monorepo, auth, orgs)
- **1** Agent & Inventory
- **2** Monitoring & Alerting
- **3** Certificates
- **4** Service Accounts
- **5** Tooling (air-gap bundlers)
- **6** Enterprise (SSO, audit)
- **7** Cloud SaaS

Infrawatch — Data Flow & Load Balancing

Vertical flow showing scalable tiers · HA deployment profile · 2× instances shown where horizontally scalable




Write · Drizzle ORM · scoped by organisation_id · consumers write to primary
DATABASE — POSTGRESQL + TIMESCALEDB · PRIMARY + STREAMING REPLICA (HA)
POSTGRESQL + TIMESCALEDB
Primary
all writes · Drizzle ORM
metrics hypertable · events spine
hosts · agents · alerts · orgs
read + write
streaming
replication
Read Replica
read-only · Next.js read queries
reduces load on primary
hot standby failover
read only
Read/Write · Server Actions via Drizzle ORM · reads from replica · writes to primary
LOAD BALANCER — DEPLOY/HAPROXY
HAProxy
HTTP mode · round-robin across Next.js instances · sticky
sessions via cookie · health-checks each
WEB APPLICATION — APPS/WEB · ×2 INSTANCES
Next.js Instance 1
Server Components · SSR
Server Actions · Zod
TanStack Query · Recharts
Better Auth (session)
horizontally scalable
Next.js Instance 2
Server Components · SSR
Server Actions · Zod
TanStack Query · Recharts
Better Auth (session)
horizontally scalable
HTTPS · authenticated session · TLS termination at HAProxy
USERS & API CLIENTS
Engineer / Admin
dashboards · alerts
inventory · config
API Client
API key auth
automation · integrations
Agent → Ingest (gRPC/mTLS)
HAProxy fan-out / fan-in
Ingest → Queue (publish)
Queue → Consumers (consume)
Consumers → DB (write)
DB ↔ Web (read/write)
Users ↔ Web (HTTPS)
horizontally scalable
Tier supports multiple instances
Infrawatch · Data Flow & Load Balancing · HA Deployment Profile · Generated 2026-03-28